Instructions

- Share git repository, Google Colab or Kaggle Notebook link with access and provide a downloadable `.ipynb` file.
- All code must execute successfully from start to finish in a fresh runtime.
- Should contain repo, notebook, source code, test cases, sample inputs/outputs,proper documentation and a brief results report .
- AI tools such as ChatGPT, Gemini, Claude, and GitHub Copilot,Codex etc are permitted.
- Clearly disclose each AI tool used, its purpose, and usage level: Low, Medium, or High.
- State your actual time spent and any external assistance received.

Question 1

Process receives records from its voice processing system. Each record contains a document ID, timestamp, and voice quality score.

Write a Python program to process the following records:

```python
documents = [
    {"id": "D101", "quality": 0.92, "time": 101 },
    {"id": "D102", "quality": 0.43, "time": 102 },
    {"id": "D103", "quality": 0.81, "time": 103 },
    {"id": "D101", "quality": 0.92, "time": 104 },
    {"id": "D104", "quality": 0.28, "time": 105 },
    {"id": "D105", "quality": 0.74, "time": 105 },
    {"id": "D102", "quality": 0.92, "time":  106 }

]
```

Tasks

1. Identify and remove duplicate document IDs.
2. Classify each unique document:
    a. Quality ≥ 0.80: **ACCEPT**
    b. Quality between 0.50 and 0.79: **REVIEW**
    c. Quality below 0.50: **RETAKE**
3. Calculate the average quality score of the unique documents.

Return the number of documents in each category.

Deliverables: Python code, sample output, and an explanation of your duplicate-handling strategy.

Question 2

A system captures some images. However, some images are blurred, poorly illuminated, rotated, or contain shadows.

Tasks

1. Explain how you would detect whether a captured document image is suitable for OCR.
2. Implement at least two image-quality checks using Python and OpenCV. Examples:
    - Blur detection using the variance of the Laplacian.
    - Brightness measurement using grayscale intensity.

Create a function that returns a result such as:

```python
{
    "is_usable": False,
    "blur_score": 18.5,
    "brightness": 72.0,
    "reason": "Image may be blurred or underexposed"
}
```

Test the function on at least three cases: a clear image, a blurred image, and a dark image.

Deliverables: Executable Python code, test cases, output, and a short explanation of the thresholds selected.

Question 3

A Process uses OCR and handwriting recognition to convert into digital text. The OCR engine may return incorrect or uncertain words.

Assume the OCR system returns:

```python
ocr_results = [
    {"text": "Paracetamol", "confidence": 0.98},
    {"text": "500 mg", "confidence": 0.96},
    {"text": "twlce daily", "confidence": 0.61},
    {"text": "for 5 days", "confidence": 0.94},
    {"text": "with", "confidence": 0.14},
    {"text": "water", "confidence": 0.94},
    {"text": "Unknown Drug", "confidence": 0.58},
    {"text": "UnknownDrug", "confidence": 0.38}
]
```

Tasks

1. Write a Python function that classifies each OCR result as:
    a. **HIGH_CONFIDENCE**: confidence ≥ 0.90
    b. **REVIEW_REQUIRED**: confidence between 0.60 and 0.89
    c. **LOW_CONFIDENCE**: confidence below 0.60
2. Implement a basic text-cleaning function without silently changing the original OCR output.
3. Design a mechanism that stores both the original text and any corrected text, along with a reason for the correction.

Explain how you would validate medicine names against a trusted medicine dictionary.

Deliverables: Python implementation, test cases, and a short explanation of how uncertain results should be handled.

Question 4

The platform must transform handwritten text into structured digital records.
Example input:

```
Patient: Rahul Sharma
Medicine: Paracetamol
Strength: 500 mg
Frequency: Twice daily
Duration: 5 days
```

Tasks
Convert the text into a JSON object with the following fields:

```json
{
  "patient_name": "Rahul Sharma",
  "medicine_name": "Paracetamol",
  "strength": "500 mg",
  "frequency": "Twice daily",
  "duration": "5 days",
  "validation_status": "REVIEW_REQUIRED"
}
```

1. Write Python validation rules to check:
    - Required fields are present.
    - Duration is expressed in an acceptable format.
    - Strength and frequency are not accidentally swapped.
    - Missing or ambiguous fields are flagged instead of guessed.
2. Handle incomplete input, such as a missing medicine name or unreadable duration.

Explain how you would use an LLM to extract fields while validating its output against a predefined schema.

Deliverables: Python code, valid and invalid examples, and sample JSON output.

Question 5

Build a complete pipeline that accepts an image containing text, extracts and validates its content, and returns accurate, structured results with minimum latency and API cost.

Criteria / Pipeline Steps

1. Image Preprocessing
1. Image Processing
2. Intelligent Routing
3. AI-Based Text Extraction
4. Accuracy and Validation
5. Cost Optimization
6. Performance Optimization
7. Structured Output

Each step must document its purpose, implementation, input/output, and impact on extraction accuracy, latency, and cost. Steps may be added, removed, or modified based on your preference.

Success Criterion: Deliver a working, tested pipeline that maximizes text extraction accuracy while minimizing end-to-end latency and LLM API cost.

Deliverables: Source code, README with setup instructions, sample inputs and outputs, a small test suite, and a brief explanation of design decisions.

Question 6

Scenario: Build a Python chatbot that answers questions from multiple independent knowledge bases. You may use company policies, software documentation, synthetic project records, or public technical documents.Use multiple Projects and each project has its own knowledge base, terms, records, sources and data.

Required features

- Multiple documents or knowledge sources.
- Question-answering with source references.
- Conversation history and follow-up questions.
- Correct topic switching.
- User- or project-level access controls.
- Handling of missing or contradictory information.
- Detection and prevention of cross-document context leakage.

Tasks

1. Implement a minimal working prototype.
2. Demonstrate five failure cases and their fixes.
3. Include automated tests for incorrect retrieval, topic switching, missing evidence, and cross-user data access.
4. Explain the architecture and the purpose of each component.
5. Document limitations and propose improvements for production deployment.

Deliverables: Source code, README, sample conversations, automated tests, and design rationale.

Question 7

Build a pipeline that transcribes legal conversations locally, removes irrelevant dialogue, extracts legally relevant information and identifies verifiable legal sections with minimum latency and cost. Support Hindi, English, and Hinglish where feasible

Requirements

1. Speech-to-Text
2 Transcript correction
3. Legal Relevance Filtering: segments filternation as `LEGALLY_RELEVANT`, `IRRELEVANT`, or `UNCERTAIN`. Preserve legally significant facts, statements, dates, agreements, allegations, evidence, and procedural details.
3. Legal Extraction: Extract parties, facts, dates, legal terms, and explicitly mentioned sections. Identify potentially applicable provisions using a versioned local legal database. Never invent legal references.
4. Local-First Processing: Use deterministic rules, legal dictionaries, local databases, and suitable ML models (CNN, RNN, or transformers). Optional LLM usage must provide measurable benefits.
5. Validation & Optimization: Verify outputs against transcript evidence; preserve negations and uncertainty. Measure accuracy, word error rate, latency, CPU/RAM usage, and API cost.

Deliverables: Executable Python code, Jupyter notebook, automated tests, and benchmark results.

Success Criterion: Deliver a working, tested pipeline that maximizes transcription and legal relevance accuracy while minimizing latency and cost.